

Abstract (+)

Vehicle

-vehicle Number: String

-vehicleType: VehicleType

+ getVehicleNumber(): String

+ getVehicleType(): VehicleType

+ getRequiredSpots(): int

+ canFitInSpot (spot: ParkingSpot): boolean

Car

+ getRequiredSpots()

+ canFitInSpot()

Motorcycle

+ get Required Spots()

+ canFitInSpot()

Bus

+ getRequired Spots()

+ canFitInSpot()

ParkingLot

- parkingFloors : List < ParkingFloor >
- entryGates : List < EntryGate >
- exitGates : List < ExitGate >
- activeTickets : Map <String, ParkingTicket>
- + allocationSpot (Vehicle vehicle) : ParkingTicket
- + releaseSpot (Ticket ParkingTicket) : void
- + getAvailableSpots : int
- + getTicket (Vehicle vehicle) : String : ParkingTicket

here parkingLot has
 - a parkingFloor, exit and
 entry gate -> which indicate
 has - a relationship

- ParkingFloor : Floor < ParkingFloor >
- EntryGate : Gate < EntryGate >
- ExitGate : Gate < ExitGate >
- Ticket : Ticket < Ticket >

↓

Parking Flood

- Flood Number: int

- parkingSpot: List < ParkingSpot >

+ findAvailableSpot (vehicle: Vehicle): ParkingSpot

+ getAvailableSpots(): List < ParkingSpot >

↓
Abstract

ParkingSpot

- spotId: int

- occupied: boolean

- spotType: SpotType

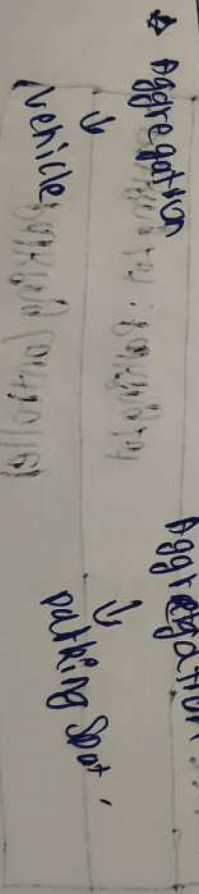
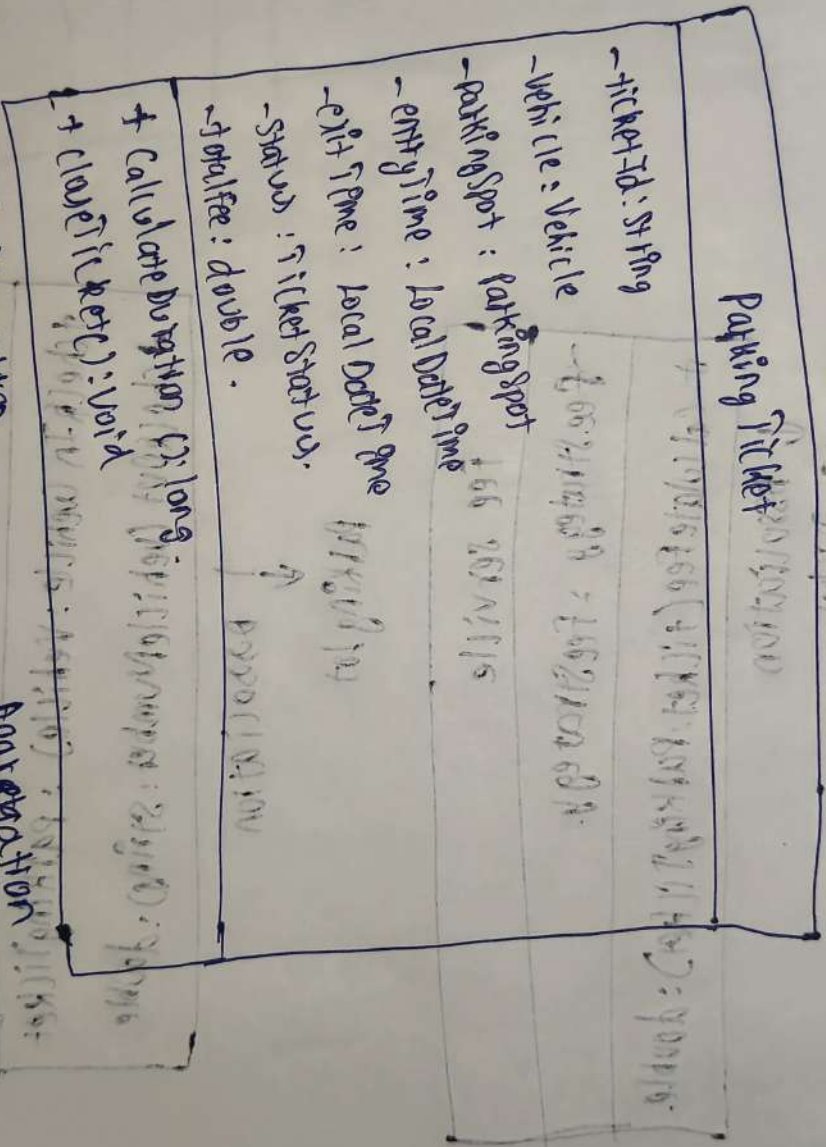
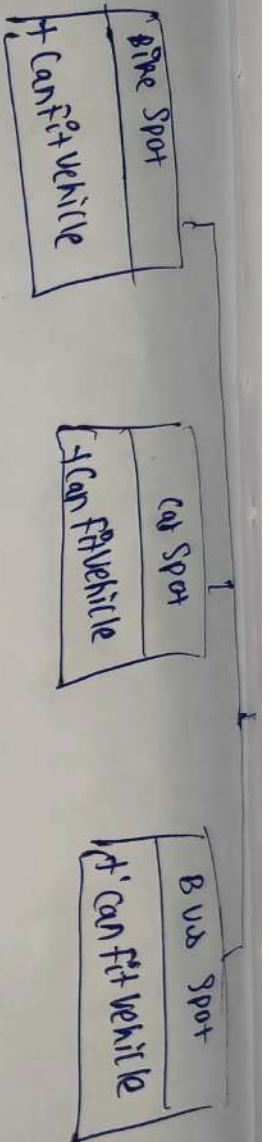
- parkedVehicle: Vehicle

+ occupy (vehicle: Vehicle): void

+ free(): void

+ isAvailable: boolean

+ canFitVehicle (vehicle: Vehicle): boolean



Parking Controller

Parking lot : Parking lot

VehicleIn Vehicle : Vehicle : ParkingTicket

VehicleOut VehicleNumber : String : double

- VehicleIn : Vehicle

- VehicleOut : Vehicle

- VehicleIn : Vehicle

- VehicleOut : Vehicle

- VehicleIn : Vehicle

- VehicleOut : Vehicle

- VehicleIn : Vehicle

- VehicleOut : Vehicle

Association

Parking lot

Fee service

Fee strategy : Fee strategy

Calculate Fee (Ticket, Parking Ticket) : double

Association

Parking Ticket

VehicleIn : Vehicle

VehicleOut : Vehicle

VehicleIn : Vehicle

VehicleOut : Vehicle

VehicleIn : Vehicle

VehicleOut : Vehicle

<< interface >>

